

Week 2 Preview — Loops, Strings in Loops, and Guess-and-Check

What we'll learn this week

This week programs learn to **repeat**. Think of Netflix bingeing a show: "are there more episodes? yes → play the next one" — that is exactly a `while` loop, and the version that auto-plays only 4 episodes is a `for` loop over a fixed sequence. We will loop over numbers and strings, and use loops to solve problems by systematically guessing and checking every candidate answer.

Terms to know

Term	One-line definition
<code>while</code> loop	Repeats its code block as long as a condition is <code>True</code> ; if the condition never becomes <code>False</code> , it loops forever
infinite loop	A loop whose condition never becomes <code>False</code> (e.g. <code>while True:</code>); stop it with CTRL-C in the shell
<code>for</code> loop	Repeats once for each value in a sequence; the loop variable takes each value in order
<code>range(start, stop, step)</code>	Generates a sequence of ints from <code>start</code> up to but not including <code>stop</code> — a lot like string slicing
<code>break</code>	Immediately exits the innermost loop it is in, skipping the rest of the block
guess-and-check (exhaustive enumeration)	Guess a value, check if it is correct, and keep guessing systematically until you find it or run out of values

Try this code before class

Taken from Lecture 4, "Guess-and-check square root with while loop" (slide 18). **Predict the output before you run it** — try entering 16, then 15.

```
guess = 0
x = int(input("Enter an integer: "))
while guess**2 < x:
    guess = guess + 1
if guess**2 == x:
    print("Square root of", x, "is", guess)
else:
    print(x, "is not a perfect square")
```

How many times does the loop body run for $x = 16$? What happens if you enter a negative number?

Questions to think about

1. Guess-and-check "can't test an infinite number of values — you have to stop at some point." How do *you* decide when to stop searching for something in real life (a lost item, an apartment, a job)?
 2. The same factorial problem can be written with a `while` loop or a `for` loop. If both work, what might make a programmer prefer one over the other?
-

Source: MIT OCW 6.100L Introduction to CS and Programming Using Python (Fall 2022), Lectures 3–4 (Ana Bell). Licensed under CC BY-NC-SA. <https://ocw.mit.edu>